

✓ Labsheet 9

```
!pip install ultralytics
```

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: opencv-python!=4.13.0.90,>=4.7.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: ultralytics-thop>=2.1.6 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: ultralytics-platform>=0.1.45 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
from ultralytics import YOLO
from google.colab import files
```

```
# Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

print("YOLOv8 model loaded successfully!")
```

YOLOv8 model loaded successfully!

```
# Upload an image
uploaded = files.upload()

# Get uploaded file name
image_path = list(uploaded.keys())[0]

print("Image uploaded successfully:", image_path)
```

bus.jpg

bus.jpg(image/jpeg) - 45642 bytes, last modified: 9/18/2026 - 100% done
Saving bus.jpg to bus.jpg
Image uploaded successfully: bus.jpg

```
# Read image
image = cv2.imread(image_path)

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display original image
plt.figure(figsize=(10, 6))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Original Image")
plt.show()
```

Original Image



```
# Perform object detection
results = model(image_path)

print("Object detection completed!")
```

```
image 1/1 /content/bus.jpg: 448x640 2 persons, 1 bus, 2 handbags, 293.0ms
Speed: 21.3ms preprocess, 293.0ms inference, 29.3ms postprocess per image at sha
Object detection completed!
```

```
# Get result image with bounding boxes
result_image = results[0].plot()

# Convert BGR to RGB
result_image = cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB)

# Display detected image
plt.figure(figsize=(12, 8))
plt.imshow(result_image)
plt.axis("off")
plt.title("YOLOv8 Object Detection")
plt.show()
```

YOLOv8 Object Detection



```
# Print detected objects
for result in results:
    boxes = result.boxes

    if boxes is not None:
        for box in boxes:
            class_id = int(box.cls[0])
            confidence = float(box.conf[0])

            class_name = model.names[class_id]

            print(
                "Object:",
                class_name,
                "| Confidence:",
                round(confidence, 2)
            )
```

```
Object: person | Confidence: 0.9
Object: person | Confidence: 0.87
Object: bus | Confidence: 0.53
Object: handbag | Confidence: 0.37
Object: handbag | Confidence: 0.3
```

```
# Count objects
object_count = {}

for result in results:
```

```
for box in result.bboxes:
    class_id = int(box.cls[0])
    class_name = model.names[class_id]

    if class_name in object_count:
        object_count[class_name] += 1
    else:
        object_count[class_name] = 1

print("Detected Objects:")
for obj, count in object_count.items():
    print(obj, ":", count)
```

```
Detected Objects:
person : 2
bus : 1
handbag : 2
```

```
# Save result image
output_image = results[0].plot()

cv2.imwrite("object_detection_result.jpg", output_image)

print("Result saved as object_detection_result.jpg")
```

```
Result saved as object_detection_result.jpg
```

QUESTIONS:

Q1. Differentiate between image classification, object detection, and image segmentation.

Answer:

Technique Meaning Image Classification Identifies the overall category of an image. Object Detection Identifies objects and their locations using bounding boxes. Image Segmentation Classifies individual pixels of an image into different regions.

For example, classification may say "Dog", detection identifies where the dog is, while segmentation identifies the exact pixels belonging to the dog.

Q2. Explain the working principle of YOLO.

Answer:

YOLO stands for You Only Look Once. It is a real-time object detection algorithm.

YOLO processes the complete image in a single pass through a neural network. It predicts:

Object location Bounding box Class label Confidence score

The detected objects are then displayed using bounding boxes.

The practical specifically asks to explain YOLO's working principle.

Q3. Compare YOLO, SSD, and Faster R-CNN. Feature YOLO SSD Faster R-CNN Speed
Very fast Fast Slower Accuracy High High Very high Real-time use Yes Yes Limited
Method Single-stage Single-stage Two-stage Applications Real-time detection
Mobile/real-time systems High-accuracy detection

YOLO and SSD are generally preferred when speed is important, while Faster R-CNN is commonly used when detection accuracy is a major priority.

The experiment asks for comparison based on speed, accuracy and applications.

Q4. What is a bounding box? Why is it important?

Answer:

A bounding box is a rectangular box drawn around a detected object.

It generally contains:

X and Y coordinates Width Height Class label Confidence score

It is important because it tells us where the object is located in the image.

Q5. Explain confidence score and IoU.

Answer:

Confidence Score

Confidence score represents how confident the model is that a particular object has been detected correctly.

For example:

Person = 0.92

means the model has a high confidence in detecting that object.

IoU

IoU stands for Intersection over Union.

It measures the overlap between the predicted bounding box and the actual bounding box.

$$\text{IoU} = \text{Area of Intersection} / \text{Area of Union}$$

Higher IoU means better overlap between the predicted and actual boxes.

Both confidence score and IoU are important measures in object detection.

Q6. Advantages of pre-trained deep learning models over traditional image processing techniques.

Answer:

Advantages include:

They can recognize complex objects. They require less manual feature engineering. They can be trained on large datasets. They provide better performance for complex images. Pre-trained models can be reused for different applications. They can detect multiple objects in a single image. Q7. Why are MS COCO and Pascal VOC widely used?

Answer:

MS COCO and Pascal VOC are popular because they provide large, standardized and labeled datasets for computer vision research.

They contain images with object annotations that can be used for:

Training models Testing models Comparing different algorithms Evaluating object detection performance

The experiment specifically mentions MS COCO, Pascal VOC and Open Images Dataset as standard datasets.

Q8. Mention five real-world applications.

Answer:

Five applications are:

Autonomous vehicles Surveillance systems Healthcare Retail analytics Industrial inspection

Other applications include traffic monitoring, face/object tracking and smart city systems.

These application areas are also listed in the practical's objectives.

Q9. What challenges are commonly encountered?

Answer:

Common challenges include:

Poor lighting Occlusion of objects Small objects Similar-looking objects Complex backgrounds Different object sizes Blurred images Multiple objects overlapping each other

These challenges can cause false detections or missed objects, which are among the evaluation points mentioned in the experiment.

Q10. How can object detection systems be improved?

Answer:

Object detection systems can be improved by:

Using larger and better-quality datasets. Using advanced deep learning architectures.

Applying image preprocessing and augmentation. Using better hardware for faster

inference. Fine-tuning pre-trained models. Improving training techniques. Using higher